The Latency Cost Of Censorship Resistance

Ittai Abraham[1], Yuval Efron[2], and Ling Ren[3]

[1]Intel Labs
[2]Columbia University
[3]University of Illinois at Urbana–Champaign

November 21, 2025

Abstract

On the road to eliminating censorship from modern blockchain protocols, recent work in consensus has explored protocol design choices that delegate the duty of block assembly away from a single consensus leader and instead to multiple parties, referred to as includers. As opposed to the traditional leader-based approach, which guarantees transaction inclusion in a block produced by the next correct leader, the multiple includer approach allows blockchain protocols to provide a strong censorship-resistance property for users: A timely submitted transaction is guaranteed to be included in the next confirmed block, regardless of the leader's behavior. Such a guarantee, however, comes at the cost of 2 additional rounds of latency to block confirmation, compared to the leader-based approach. Is this cost necessary?

We introduce the *Censorship Resistant Byzantine Broadcast* (CRBB) problem, a one-shot variant that distills the core functionality underlying the multiple-includer design paradigm. We then provide a full characterization, both in synchrony and partial synchrony, of the achievable latency of CRBB in executions with a correct leader, which is the most relevant case to practice. Our main result is an inherent latency cost of two additional rounds compared to the classic Byzantine Broadcast (BB) problem. For example, synchronous protocols for CRBB require 4 rounds whenever BB requires 2 rounds. Similarly, up to a small constant in the resilience, partial synchrony protocols for CRBB require 5 rounds whenever BB requires 3 rounds.

1 Introduction

The problem of censorship [5] has been a dark cloud trailing blockchain protocols since their inception, with its shadow assailing the vast majority of deployed blockchains. In many aspects, the problem can be traced to the monopoly power over block production which arises in *leader-based* consensus protocols. The consensus task requires n parties, of which f are faulty and controlled by an *adversary*, to maintain a consistent and live log of incoming transactions. Roughly, in leader-based protocols, parties take turns serving as leaders. The leader has total control over the content of the next block of transactions. A convenient way of viewing the leader-based approach to consensus is as a series of sequential and layered instances of the well-studied Byzantine Broadcast (BB) problem, in which a designated leader is responsible for broadcasting a value to all other parties. The leader-based paradigm is highly attractive, as it allows the design of simple and concretely efficient consensus protocols [7, 23, 4, 1, 17, 20, 21, 16, 10, 8, 9]. However, with temporary

monopoly power over the contents of a block, a leader has the ability to censor, reorder, or insert transactions of their own to the next block, subverting fair market conditions, and extracting substantial economic value away from the chain [3], a phenomenon colloquially referred to as *miner extractable value* (MEV). Several proposals in the literature aim to mitigate the ability of a leader to engage in such behavior, thus endowing protocols with some notion of *censorship resistance*.

Censorship resistance. When discussing censorship resistance, it would be prudent to keep in mind the notion of a client, disjoint from the validators tasked with reaching consensus. A client holds a transaction, and wishes to land this transaction in the chain, i.e., to have it included in a block. A bare minimum notion of censorship-resistance is that of *eventual inclusion*, which merely guarantees that the client's transaction will *eventually* be included in *some* future block. Such a guarantee is quite weak in practice: Most, if not all, leader-based consensus protocols in the literature only guarantee that a transaction is included in a block once a correct leader has been elected, which in the worst case can be $f + 1$ blocks in the future. Many applications require a stronger notion of censorship resistance. Ideally, we would like to have a guarantee that a timely submitted transaction be included in the very *next* confirmed block. This strong notion is the one we are interested in for this paper.

Definition 1.1 (Censorship-resistance, Informal)**.** *A consensus protocol is censorship-resistant if for any client submitting a transaction tx at time t, tx is guaranteed to be included in the* first *block confirmed at time $\geq t$, if not before.*

Recent work in the literature has explored alternative design choices that obtain such a guarantee, specifically those that rely on *multiple includers* to determine the contents of each block [22, 6, 13, 19, 14, 12, 15]. While these approaches delegate a significant portion of block assembly to includers, a leader is still tasked with driving consensus forward in a similar manner to the leader-based approach, to inherit its efficiency and simplicity.

Censorship resistance vs. latency. Upon inspection, one can perhaps note that the above notion is pointless, unless paired with some guarantee on the *latency* in which blocks (and hence transactions) are confirmed. After all, the notion of a block is flexible, and one can always treat the BB instances of a group of $f + 1$ leaders as one block, guaranteeing that all clients be included in the instance of the correct leader, if nothing else. The worst-case latency is arguably the wrong metric to look at, as it is $\Omega(f)$ both in the multiple-includer and in leader-based approaches. A far more interesting notion to consider in this regard is that of *good-case* latency, i.e., the latency to confirmation in the case where the next leader is correct, which is the common case in practice. Interestingly enough, the traditional leader-based design still has an edge over the multiple-includer approach when it comes to good-case latency. Specifically, leader-based protocols, in which the leader has total monopoly over block contents (and thus do not guarantee the strong censorship resistance property of Definition 1.1), boast good-case latency of 2 and 3 rounds, in synchrony and partial-synchrony, respectively. This is contrasted with the multiple-includer approach, which in its most optimized implementation to date [15], has good-case latency of 4 and 5 rounds, in synchrony and partial-synchrony, respectively. It would be ideal if one could achieve the best of both worlds: A protocol obtaining censorship resistance of the multiple-includer approach, with good-case latency matching the classic leader-based approach.

Main result. In this paper, we resolve this question in the negative. We observe a fundamental good-case latency cost for achieving censorship resistance in consensus, morally exhibiting that a protocol satisfying Definition 1.1, must have good-case latency of at least 4 and 5, in synchrony and partial synchrony, respectively. In other words, we show that in order to provide censorship resistance, at least two extra rounds of latency must be incurred, even in executions with a correct leader (which is the setting where efficiency matters the most for leader-based consensus).

Theorem 1.2 (Main theorem, Informal)**.** *Any censorship resistant consensus protocol must have good-case latency of* 4 *in synchrony, and* 5 *in partial synchrony.*

We formalize the above statement by introducing and examining the *censorship-resistant byzantine broadcast* (CRBB) problem, a one-shot version which distills the core functionality underlying the multiple-includer approach.

Censorship-Resistant Byzantine Broadcast. The classic formulation of the Byzantine Broadcast (BB) problem designates a leader ℓ amongst n parties, that holds an input value v, which can be thought of as the leader's proposed next block. How ℓ acquires/constructs its input is treated as exogenous to the problem. To incorporate the fact that a leader's block proposal consists of transactions submitted by entities *external* to the protocol, we incorporate an additional type of protocol participant: *clients*. In CRBB, clients hold input values, and each correct party outputs a set of values S. The traditional validity property of BB is replaced by the **censorship resistance** property: If a client is correct, its input value must be included in the output set of correct parties. We believe that the CRBB problem formally captures the censorship resistance notion desired in the various multiple-includer blueprints in the literature. In particular, protocols following the multiple-includer blueprint can be cast as protocols solving the CRBB problem, and a solution to the CRBB problem can also be adapted into a censorship-resistant blockchain.

Efficiency. At this point, the leader's ℓ role might seem unclear, as it no longer has the input. As we discussed previously, however, practical protocols that adopt the multiple-includer approach still rely on the leader for *efficiency*. We formally capture this by studying the *decision* latency of the CRBB problem in executions in which the leader ℓ is correct. Such a latency bound is known in the literature as good-case latency [2, 11]. A beneficial way of thinking about good-case latency is as serving as a good proxy to the block interval in a blockchain protocol. In its most optimized implementation [15], the multiple-includer approach incurs a cost of two additional rounds to every view of a given consensus protocol. This in particular translates to an added two rounds on the good-case latency. For example, in synchrony, a protocol for BB with good-case latency of 2 can be turned into a CRBB protocol with good-case latency of 4. Similarly, in partial synchrony, a protocol with good-case latency of 3 can be turned into a CRBB protocol with good-case latency of 5. The main question we study in this paper is whether any protocol for CRBB must pay this added cost of two additional rounds compared to its BB counterpart, as concrete latency is of the utmost importance to modern blockchains, aiming to support financial applications that require settlement times in the order of hundreds of milliseconds.

With CRBB in mind, we can now more formally state out main results. In particular, we prove the following.

Theorem 1.3 (See Theorem 3.1)**.** *Any protocol for* CRBB *in synchrony requires* 4 *rounds, even in executions with a correct leader.*

Theorem 1.4 (See Theorem 4.1)**.** *Any protocol for* CRBB *in partial synchrony secure against more than* $\frac{1}{5}$*-fraction of byzantine faults requires* 5 *rounds, even in executions with a correct leader.*

Furthermore, we provide complementary upper bounds, establishing the tightness of our lower bounds result. Specifically, we prove the following.

Theorem 1.5 (See Theorem 5.3)**.** *There exists a protocol for* CRBB *secure in synchrony against up* $\frac{1}{2}$*-fraction of byzantine faults, with decision latency of* 4 *in executions with a correct leader.*

Theorem 1.6 (See Theorem 5.2)**.** *There exists a protocol for* CRBB *secure in partial synchrony against up* $\frac{1}{5}$*-fraction of byzantine faults, with decision latency of* 4 *in executions with a correct leader.*

Theorem 1.7 (See Theorem 5.1)**.** *There exists a protocol for* CRBB *secure in synchrony against up* $\frac{1}{3}$*-fraction of byzantine faults, with decision latency of* 5 *in executions with a correct leader.*

Outline of the paper. In Section 2, we formally describe our model that augments the traditional model of consensus with clients, we formally define the CRBB problem, the notions of decision latency and good-case latency. In Section 3, we formally state and prove the 4 round lower bound for CRBB in synchrony. In Section 4, we formally state and prove the 5 round lower bound for CRBB in partial synchrony. In Section 5, we formally state and prove our complementary upper bounds.

2 Model & Definitions

The model. We consider a permissioned setting of n validators $p_1, \ldots, p_n$, with an established Public Key Infrastructure (PKI). The protocol has additional parties: a set $\mathcal{C}$ of *clients*, whose size is determined by the adversary at the beginning of the protocol and is revealed to all validators. We assume that each client has a public/private key pair which becomes known to the validators prior to the beginning of the protocol. We assume pair-wise communication channels between any pair of validators and any pair of validator and client. We consider a universe I of input values.

Timing model. For simplicity, we assume that all validators have synchronized local clocks. As we are proving lower bounds, this only serves to strengthen our results. We consider two timing models in this work: Synchrony and partial synchrony. In synchrony, there is a globally known Δ such that a message sent at time t is delivered by time $t + \Delta$. Whenever we discuss synchrony, we assume that $\Delta = 1$ for ease of exposition. Again, as we are proving lower bounds, this only strengthens our result. In partial synchrony, there is an adversarially chosen GST, such that a message sent at time t must be delivered by time $\max\{GST + \Delta, t + \Delta\}$. Within the bounds of these constraints, message delivery times are to the discretion of the adversary.

Adversary. In this work, we consider two two fault types: Byzantine and omission. A Byzantine party secedes the entirety of its internal state to the *adversary* and is controlled by the adversary throughout the protocol. A party suffering from an omission failure can have its incoming and outgoing messages omitted from the network, and the party is *unaware* whether its messages have been omitted. The adversary has a bird's eye view of the incoming and outgoing messages between

parties and can arbitrarily corrupt parties of choice and omit from the network both incoming and outgoing messages from said parties, but it has no control over the internal state of parties experiencing omission failures. We say a party is *corrupt* if it suffers from either an omission fault or a Byzantine fault. Note that the above discussion refers to parties, which include both validators and clients.

We consider deterministic protocols. As such, a protocol Π and adversary $\mathcal{A}$ determine the execution of the protocol, i.e., all messages sent by all correct parties at all times, the internal states of all correct parties at any time in the protocol, the behavior of corrupt parties, and message delivery timings. Specifically, given a protocol Π, an execution unfolds as follows: The adversary $\mathcal{A}$ chooses a finite subset $C \subseteq \mathcal{C}$ of clients to participate in the execution and chooses their inputs from I. The identities of the participating clients C are then revealed to all validators. For each client $c \in C$, the adversary chooses whether or not to corrupt it and, if so, its behavior throughout the execution. Then, $\mathcal{A}$ chooses a set of validators $F_{\mathcal{A}} \subseteq [n]$ to corrupt and their behavior throughout the execution. We say that $\mathcal{A}$ is f-bounded if $|F_{\mathcal{A}}| \leq f$. Note that the adversary's corruption bound only counts corrupt *validators*, and does not count corrupt clients.

Cryptographic primitives. We use a digital signature scheme on top of our PKI setup. In this work, we consider only computationally bounded adversaries. In particular, the adversary $\mathcal{A}$ cannot forge messages on behalf of non-Byzantine parties.

Definition 2.1. *In the* Censorship-Resistant Byzantine Broadcast *(*CRBB*) problem there are n validators $p_1, \ldots, p_n$, of which one is designated as the* leader ℓ*. Furthermore, there are m* clients *$c_1, \ldots, c_m$. Each client has an input value v_i from a universe I and each validator p_i produces an output set $S \subset I$. The following properties pertain to a given protocol Π for the* CRBB *problem.*

- ***Agreement.*** *All non-faulty validators p_i output the same subset S from Π.*
- ***Censorship resistance.*** *If the network is synchronous, or if GST=0, then for all correct clients c_i, it holds that $\langle v_i \rangle_{c_i} \in S$.*
- ***Termination.*** *All correct validators eventually decide and terminate.*

We say a protocol Π for the CRBB *is f-secure if it satisfies all the above properties against any f-bounded adversary.*

Decision Latency. For an f-bounded adversary $\mathcal{A}$ and a f-secure protocol Π for CRBB, we denote by $\mathsf{DL}_{\mathcal{A}}(\Pi)$ the smallest round R for which every correct validator *decides* (but not necessarily terminates) by round R, in the presence of $\mathcal{A}$. We refer to $\mathsf{DL}_{\mathcal{A}}(\Pi)$ as the *decision latency* of Π with respect to $\mathcal{A}$. For a family $\mathcal{F}$ of f-bounded adversaries, we define the decision latency of Π with respect to $\mathcal{F}$ to be $\mathsf{DL}_{\mathcal{F}}(\Pi) = \sup_{\mathcal{A} \in \mathcal{F}} \mathsf{DL}_{\mathcal{A}}(\Pi)$.

Good-Case Latency. The efficiency metric we focus on in this work is that of *good-case* decision latency T_{gc}. Intuitively, this metric considers the decision latency of a protocol Π in executions in which certain favorable conditions hold, namely ℓ being correct for CRBB and a partially synchronous network happens to be synchronous. More formally, we define good-case latency as follows.

1. CRBB in synchrony: For a value f, and an f-secure protocol Π for CRBB in synchrony, consider the family $\mathcal{F}_{\mathsf{gc}}$ of f-bounded adversaries $\mathcal{A}$ for which the leader ℓ is correct, i.e., $\ell \notin F$. We define the good-case latency of Π to be $\mathsf{T}_{\mathsf{gc}} \triangleq \mathsf{DL}_{\mathcal{F}_{\mathsf{gc}}}(\Pi)$.

2. CRBB in partial synchrony: For a value f, and an f-secure protocol Π for CRBB in partial synchrony, consider the family $\mathcal{F}_{\mathsf{gc}}$ of f-bounded adversaries $\mathcal{A}$ for which the leader ℓ is correct, i.e., $\ell \notin F$, and $GST = 0$. We define the good-case latency of Π to be $\mathsf{T}_{\mathsf{gc}} \triangleq \mathsf{DL}_{\mathcal{F}_{\mathsf{gc}}}(\Pi)$.

3 CRBB lower bound in synchrony

In this section, we prove the following.

Theorem 3.1. *Any protocol* Π *for the* CRBB *problem in synchrony that is* f*-secure for* $f \geq 3$ *must have* $\mathsf{T}_{\mathsf{gc}} \geq 4$.

In fact, we prove Theorem 3.1 by proving a lower bound for a significantly simplified version of the CRBB problem, in which there is only a single *client*, for which we must guarantee censorship resistance. We call this simplified variant the *client-leader* problem.

Definition 3.2. *In the* client-leader *(*CL*) problem there are* n *validators, of which one is a designated leader* ℓ*, and a single client* c*. The client has an input* $b \in \{0, 1\}$*, and all correct validators produce an output bit. The following properties pertain to a given protocol* Π *for the* CL *problem.*

- ***Agreement.*** *All correct validators output the same bit* b' *from* Π.
- ***Censorship resistance.*** *If the client* c *is correct, and the network is synchronous or* $GST = 0$*, then* $b' = b$.
- ***Termination.*** *All correct validators eventually terminate.*

We say a protocol Π for the CL problem is f-secure if it satisfies all the above properties against any f-bounded adversary. We establish Theorem 3.1 by proving the following.

Lemma 3.3. *Any protocol* Π *for the* CL *problem in synchrony which is* f*-secure for* $f \geq 3$ *must have* $\mathsf{T}_{\mathsf{gc}} \geq 4$.

Clearly, the client-leader problem CL is an easier than the general version of CRBB with an arbitrary number of clients. Thus, our 4-round lower bound for CL applies to CRBB.

Before presenting the proof, we require some definitions. We first define the notion of configurations, and *almost same* configurations.

Definition 3.4 (Configuration)**.** *For a protocol* Π*, an adversary* $\mathcal{A}$*, and a round* r*, we define the configuration at round* r *to be the tuple of internal states of all parties at the beginning of round* r *in the execution induced by* $(\Pi, \mathcal{A})$*. In case of partial synchrony, a configuration at round* r *additionally includes the set of undelivered messages at the beginning of round* r*. When the adversary is clear from the context, we denote the configuration at round* r *by* C_r.

We can now define the notion of *failure free decision* of a given configuration C.

Definition 3.5 (Failure free decision)**.** *Let* Π *be an* f*-secure protocol for* CL*, and let* $\mathcal{A}$ *be an* f*-bounded adversary,* r *be a round, and* C_r *be the induced configuration at round* r *by* $(\Pi, \mathcal{A})$*. We denote by* $\mathsf{FFV}(C_r)$ *the failure free decision value, which is the value decided by correct validators in the execution* E *for which the following holds.*

- E *extends* C_r*.*
- *In all rounds* $\geq r$*, no party experiences further omissions or other faults.*

For a configuration C_r and a correct party p, we denote by $\mathsf{state}_p(C_r)$ the internal state of party p in C_r. Note again that a party can be either a validator or a client.

Definition 3.6 (Almost same configurations)**.** *Let* Π *be a protocol,* r *be a round, and let* $\mathcal{A}, \mathcal{A}'$ *be adversaries that induce configurations* C_r, C'_r *at round* r*. We say that* C_r, C'_r *are almost same if there exits a unique party* p *s.t.* $\mathsf{state}_p(C_r) \neq \mathsf{state}_p(C'_r)$*, and we call the party* p *the* pivotal *party.*

Definition 3.7 (Almost same but failure free different (AS-FFD) configurations)**.** *If two almost same configurations* C_r, C'_r *have* $\mathsf{FFV}(C_r) \neq \mathsf{FFV}(C'_r)$*, we say they are almost same but* failure free different*.*

We are now ready to prove the following lemma.

Lemma 3.8. *Let* Π *be an* f*-secure protocol for* CL*, let* r *be a round and let* C_r, C'_r *be two almost same but failure free different configurations of* Π *at round* r*. Then, by corrupting (via an omission failure) at most a single party at round* r*, one can extend* C_r, C'_r *into round* $r+1$ *executions* D_r, D'_r *respectively that are almost same but failure free different.*

Proof. Let p be the pivotal party implied by C_r, C'_r being almost same. Consider the configuration D_{r+1} (D'_{r+1}) that extends C_r (C'_r) in which party p experiences an omission failure at the beginning of round r, and sends no messages to any party (except itself) in that round. The rest of the parties experience no faults. Note that D_{r+1}, D'_{r+1} are almost same round $r+1$ configurations, with p being the pivotal party. If $\mathsf{FFV}(D_{r+1}) = \mathsf{FFV}(C_r)$ and $\mathsf{FFV}(D'_{r+1}) = \mathsf{FFV}(C'_r)$ then $\mathsf{FFV}(D_{r+1}) \neq \mathsf{FFV}(D'_{r+1})$ and the proof is concluded.

Otherwise, assume w.l.o.g. that $\mathsf{FFV}(D_{r+1}) \neq \mathsf{FFV}(C_r)$. Consider now the round $r+1$ configurations $C_{r+1}[i]$ for $i \in [n]$, where $C_{r+1}[i]$ is obtained by omission failing party p at round r, and having p send its round r message to the parties $p, p_1, \ldots p_i$ with $p_0 = p$, and omitting the messages for the rest. Observe that $C_{r+1}[0] = D_{r+1}$. Observe further that $C_{r+1}[n-1]$ is the 1-round extension of the failure free extension of C_r. Thus $\mathsf{FFV}(C_{r+1}[0]) \neq \mathsf{FFV}(C_{r+1}[n-1])$. Thus, there exists $i \in [n-2]$ s.t. $\mathsf{FFV}(C_{r+1}[i]) \neq \mathsf{FFV}(C_{r+1}[i+1])$. Note that for all $j \in [n-2]$, $C_{r+1}[j]$, $C_{r+1}[j+1]$ are almost same, as all parties receive the same set of messages except for p_{j+1}. Thus we have that $C_{r+1}[i]$, $C_{r+1}[i+1]$ are round $r+1$ executions that are almost same but failure free different. This concludes the proof. □

Next, we prove the following.

Lemma 3.9. *Let* Π *be an* f*-secure protocol for* CL*, let* r *be a round and let* C_r, C'_r *be two almost same but failure free different configurations of* Π *at round* r*. Suppose further that* C_r, C'_r *are reachable via corrupting at most* $t < f$ *parties. Then, there exists an execution of* Π *in which not all correct validators decide by round* $r+1$*.*

Proof. Let p be the pivotal party implied by C_r, C'_r being almost same. Let b be the value decided by correct validators in round $r+1$ in which p has all of its outgoing round r messages omitted (that requires corrupting party p, if it was not already corrupted). Assume w.l.o.g. that $b \neq \mathsf{FFV}(C_r)$. Let j be some correct validator and consider the extension of C_r in which p delivers its round r message to j, and has its messages omitted to all other parties. From j's point of view, this is indistinguishable from the failure free extension of C_r, and so it decides $\mathsf{FFV}(C_r)$. From the point of view of all other correct parties (there is at least one such party due to the assumption that $f < n-1$), this is indistinguishable from the extension of C_r in which p has all of its outgoing round r messages omitted, and so they decide b. Hence, an agreement violation occurs. This concludes the proof. □

We now tend to proving our main theorem.

Proof of Theorem 3.1. Let Π be a protocol for CL, and assume towards a contradiction that it has good-case latency of $\mathsf{T}_{\mathsf{gc}} \leq 3$. Denote the parties by $c, \ell, p_1, \ldots, p_{n-1}$. Consider the round-two configuration D^0, D^1 (i.e., the states of all players after the delivery of round-one messages) where in D^b the client c has input b and experiences an omission failure at round 1 and has all of its outgoing messages be omitted for that round. If $\mathsf{FFV}(D^0) \neq \mathsf{FFV}(D^1)$ then we have obtained a pair of round-two configurations which are AS-FFD with the pivotal party being the client. Otherwise, w.l.o.g. let $0 = \mathsf{FFV}(D^0)$ be the decided value in the failure free extension of D^0. Consider now the round-two configurations $D_0, \ldots, D_{n-2}$ in which the following holds for $i \in \{0, 1, \ldots, n-2\}$:

- c has input 1. All round-one messages by all parties except for c are delivered without fault.
- c's outgoing message to ℓ, if any, is omitted.
- c delivers its round-one message to $p_1, \ldots p_i$, and has it omitted for the remaining parties.

Note that the failure free extension of D_{n-2}, from the point of view of every party in $\{p_1, \ldots, p_{n-2}\}$, is indistinguishable from the failure free extension of the following round 1 configuration D'.

- c is correct and has input 1. All round-one messages by all parties are delivered, except for the message, if any, from c to ℓ, which we omit via an incoming omission failure to ℓ.

From censorship resistance, we have that $\mathsf{FFV}(D') = 1$. From the above indistinguishability argument we get that $\mathsf{FFV}(D_{n+2}) = \mathsf{FFV}(D')$. Thus we have that $0 = \mathsf{FFV}(D_0) \neq \mathsf{FFV}(D_{n-2}) = 1$. Thus there exists $i \in \{0, \ldots, n-3\}$ such that $\mathsf{FFV}(D_i) \neq \mathsf{FFV}(D_{i+1})$. Further note that D_i, D_{i+1} are almost same, as only the state of p_i is different between the configurations. By construction of $D_0, \ldots, D_{n-2}$, we have that $p_i \neq s$. We have thus obtained, with the use of an omission failure on the client c, a pair of round two configurations D_i, D_{i+1} that are AS-FFD, in which the pivotal party is not the leader ℓ. Invoking Lemma 3.8, we obtain, with the use of at most a single additional omission failure on p_i, a pair of round three configurations E, E' that are AS-FFD. Denote by p_j the pivotal party exhibited by E, E'. There are two cases.

- $p_j \neq \ell$. Then invoking Lemma 3.9 on E, E', which are a pair round three configurations, we get that by corrupting p_j one can exhibit an execution, extending either E or E', in which a correct party does not decide by the end of round three. This contradicts the good case latency $\mathsf{T}_{\mathsf{gc}} \leq 3$ assumption, as E, E' are configurations reachable by only corrupting c and p_i, neither of which is ℓ. As $p_j \neq \ell$, corrupting p_j keeps ℓ correct, and thus all correct validators must decide after 3 rounds, contradicting Lemma 3.9.

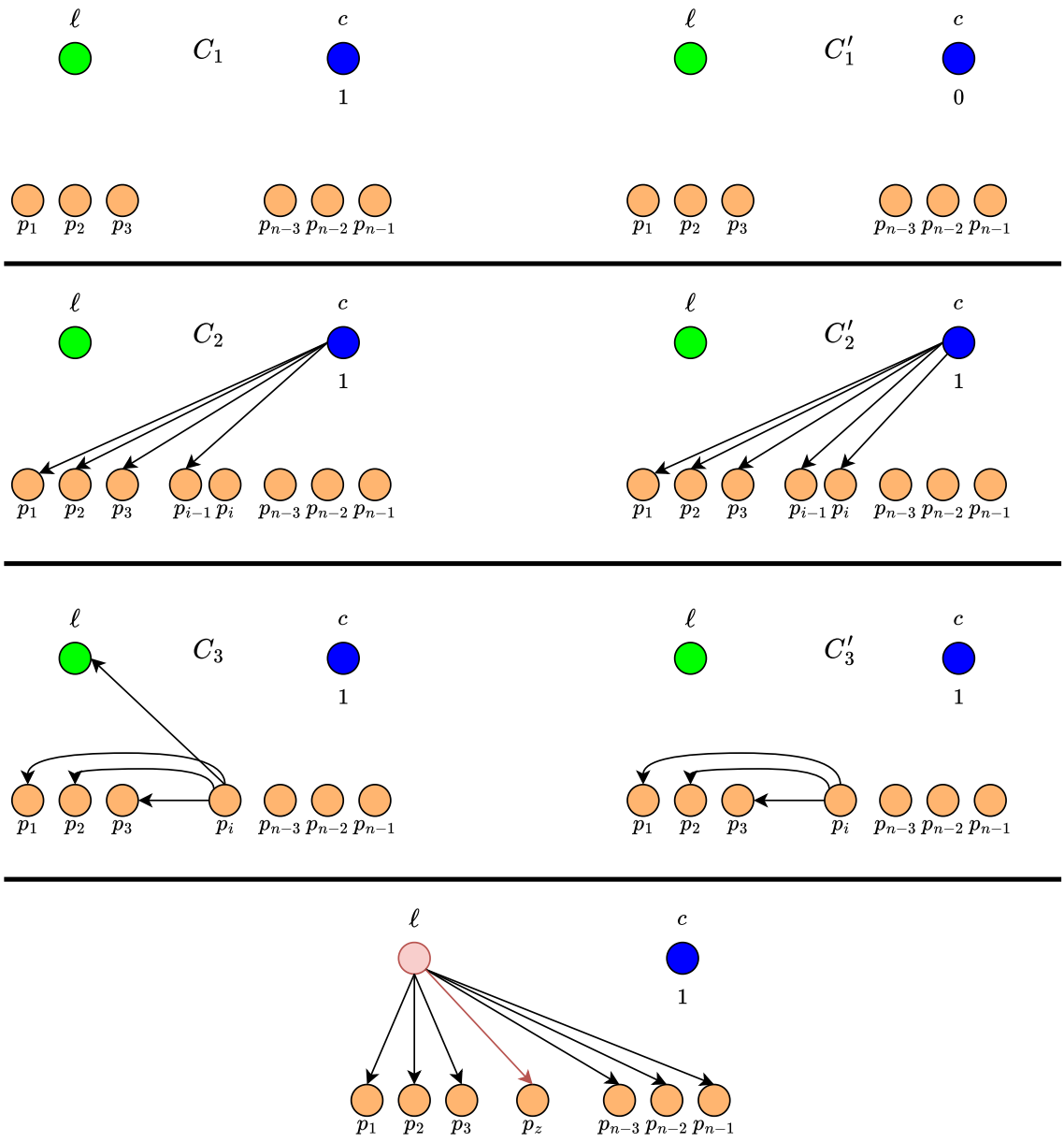

Figure 1: The critical path of the AS-FFD pairs of configuration realizing the proof of Theorem 3.1. From top to bottom we have the pair of round 1 AS-FFD configurations C_0, C_0' with pivotal party being the client c. Followed by the pair of round 2 AS-FFD configurations C_2, C_2' with pivotal party $p_i \neq \ell$, and the pair of round 3 AS-FFD configurations C_3, C_3' with pivotal party ℓ. The proof then concludes by corrupting ℓ and letting it equivocate to a particular victim validator p_z.

- $p_j = \ell$. Here we make use of a Byzantine fault, and corrupt ℓ with a Byzantine fault. Consider configurations E, E'. By the construction in Lemma 3.8, they differ from one another on whether ℓ received its round two message from p_i. Consider configuration E and let p_z be some correct validator in E. We now have ℓ do the following.
  - Send a round-three message to p_z as in the failure free extension of E', i.e. as if ℓ did not hear the round two message of p_i.
  - To all other correct parties, send a round-three message as in the failure free extension of E.

  From p_z's point of view, the execution is indistinguishable from the failure free extension of E', and thus, since E' has ℓ being correct, p_z decides $\mathsf{FFV}(E')$ at the end of round three by the good-case latency $\mathsf{T}_{\mathsf{gc}} \leq 3$ assumption. From the point of view of all other correct validators, the execution is indistinguishable from the failure free extension of E. Since ℓ is correct in E, all other correct validators decide, by the good-case latency $\mathsf{T}_{\mathsf{gc}} \leq 3$ assumption, the value $\mathsf{FFV}(E)$ by the end of round three. This is a violation of agreement as $\mathsf{FFV}(E) \neq \mathsf{FFV}(E')$. This concludes the proof.

□

4 CRBB lower bound in partial synchrony

In this section, we prove the following theorem.

Theorem 4.1. *Any protocol* Π *for the* CRBB *problem in partial synchrony which is* f*-secure for* $5f \geq n + 6$ *must have good case latency at least* $\mathsf{T}_{\mathsf{gc}} \geq 5$.

In a similar fashion to Section 3, we establish Theorem 4.1 via a lower bound on the CL problem. Specifically, we prove the following.

Lemma 4.2. *Any protocol* Π *for the* CL *problem in partial synchrony which is* f*-secure for* $5f \geq n + 6$ *must have good case latency at least* $\mathsf{T}_{\mathsf{gc}} \geq 5$.

The intuition. Apriori, it would be tempting to follow the same approach as in Theorem 3.1, by identifying a pivotal party in a pair of almost same but failure free different (AS-FFD) pairs of executions for each round, and carefully arguing that the leader isn't the pivotal party in the earlier rounds of this argument. Specifically, we observed in the proof of Theorem 3.1, that even in synchrony, one can construct round 0 and 1 configurations C_0, C_0', C_1, C_1' which are AS-FFD, and in which the pivotal party is not ℓ. However, this argument cannot be extended to round 2, and indeed, arguably any protocol worth its salt better rely on the leader for early deciding as heavily as possible. In Theorem 3.1, one can deal with the case of ℓ being pivotal thanks to the fact that there is a single round left prior to the early deciding condition kicking in. As such, corrupting ℓ and letting it equivocate causes an agreement violation. This approach fails when a 5-round lower bound is desired, as after the equivocation, there is still an additional round left, in which parties can detect ℓ's misbehavior, thus relinquishing the early deciding path. At this point, we take inspiration from the work of [2], which shows early deciding with just a leader requires 3 rounds in partial synchrony and in the presence of $5f \geq n$ byzantine faults. We adapt and generalize their

tools in order to make them applicable to our setting, in which the leader ℓ is the pivotal party for the round 2 AS-FFD configurations, allowing us to show that at least 3 rounds are required from that position.

Proof of Theorem 4.1. Let Π be an f-secure protocol for CL in partial synchrony with $5f - 6 \geq n$ and assume towards a contradiction that it has good case latency of $\mathsf{T_{gc}} \leq 4$. Denote the parties by $c, \ell, p_1, \ldots, p_{n-1}$. Denote the round two configurations D^0, D^1 (i.e. the states of all parties after the delivery of all non delayed round one messages) where D^b results from omitting all of the client's round 1 messages when the client has input b, and delivering all other messages within Δ time. If $\mathsf{FFV}(D^0) \neq \mathsf{FFV}(D^1)$[1], then we have identified two AS-FFD round two configurations with the pivotal party being the client. Otherwise, assume w.l.o.g. that $\mathsf{FFV}(D^1) = 0$ and denote $D^1 = D$. Consider now the round two configurations $D_0, \ldots D_{n-1}$ in which the following holds for $i \in \{0, \ldots, n-1\}$:

- GST is chosen to be sufficiently large.
- c has input 1. All round one messages by all parties except for c are delivered without fault within Δ time.
- c's outgoing message to ℓ, if any, is omitted.
- c has its round 1 messages to $p_1, \ldots, p_i$ delivered within Δ time and the messages to the remaining parties are omitted.

Note that $D_0 = D$. Note further that the failure free extension of D_{n-1}, from the point of view of every party in $p_1, \ldots, p_{n-1}$, is indistinguishable from the failure free extension of the following round 1 configuration D'.

- GST$= 0$.
- c has input 1. All messages by all parties are delivered in a timely manner, except for messages, if any, from c to ℓ, which we omit via an omission failure on ℓ.

From censorship resistance, we have that $\mathsf{FFV}(D') = 1$. From the above indistinguishability argument, we further get that $\mathsf{FFV}(D_{n-1}) = \mathsf{FFV}(D')$. We can conclude that $0 = \mathsf{FFV}(D) \neq \mathsf{FFV}(D_{n-1}) = 1$. Thus there exists $i \in \{0, \ldots, n-2\}$ such that $\mathsf{FFV}(D_i) \neq \mathsf{FFV}(D_{i+1})$. Further note that D_i, D_{i+1} are almost same, as only the state of p_i is different between the configurations. By construction of $D_0, \ldots, D_{n-1}$, we have that $p_i \neq \ell$. We have thus obtained, using an omission failure on c, two round two configurations D_i, D_{i+1} which are AS-FFD, in which the pivotal party is not the leader ℓ. Invoking Lemma 3.8 we obtain, with only omitting round two messages of p_i, two round three configurations C_3, C_3' which are almost AS-FFD with pivotal party p_j. We now fork into two cases.

- $p_j \neq \ell$. This is the easy case. We invoke Lemma 3.8 again, using only an omission on p_j's messages to obtain two round four configurations F, F' which are AS-FFD with pivotal party p_k. Whether $p_k = \ell$ or not, the proof is then concluded using the same two case analysis as in Theorem 3.1.

[1]The failure free extension refers to the extension in which no further messages are delayed and no party gets corrupted

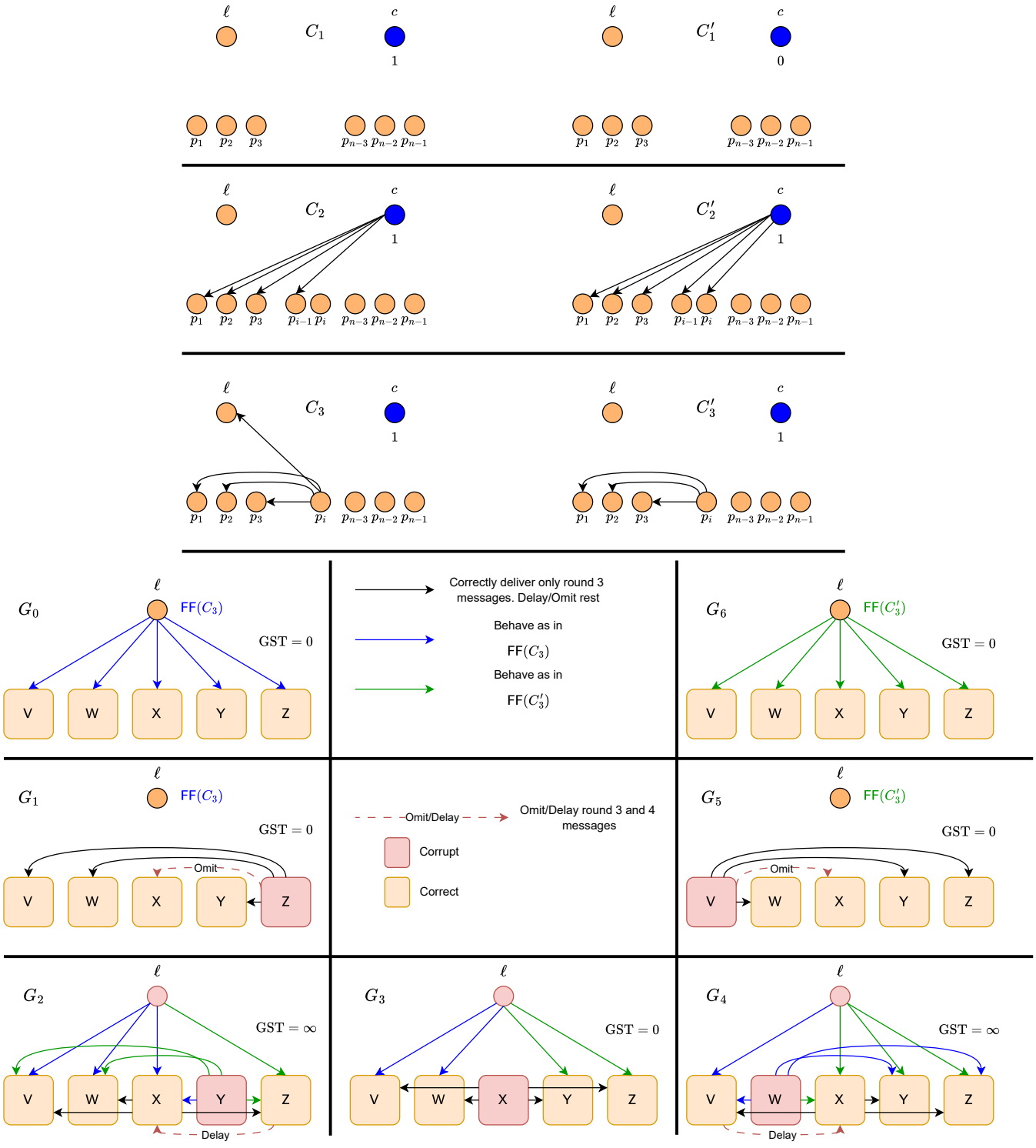
C_1
C'_1
C_2
C'_2
C_3
C'_3
ℓ
c
1
0
p_1 p_2 p_3 p_{i-1} p_i p_{n-3} p_{n-2} p_{n-1}
G_0
FF(C_3)
GST $= 0$
V W X Y Z
Correctly deliver only round 3 messages. Delay/Omit rest
Behave as in FF(C_3)
Behave as in FF(C'_3)
G_6
FF(C'_3)
G_1
Omit
G_5
Omit/Delay
Omit/Delay round 3 and 4 messages
Corrupt
Correct
G_2
GST $= \infty$
Delay
G_3
G_4

Figure 2: (Previous page) Pictorial representation of the critical path of configurations and executions defined during the proof of Theorem 4.1. From top to bottom: The pair of round 1 AS-FFD configurations C_1, C_1' with pivotal party being the client c. Followed by the pair of round two AS-FFD configurations C_2, C_2' with pivotal party $p_i \neq \ell$. From here, an invocation Lemma 3.8 provides a pair of round three AS-FFD configurations C_3, C_3', for which the interesting case is the one with ℓ being pivotal. What follows is the description of seven executions that extend either C_3 or C_3'. The rectangles labeled V, W, X, Y, Z represent the partition of parties into five sets as in the proof. Black arrows between two sets indicate correct delivery of round three messages in the direction of the arrow, and delivery of no further messages, via network delays or omission failures for correct and corrupt parties, respectively. Blue and green arrows indicate correct behavior as in $\mathsf{FF}(C_3), \mathsf{FF}(C_3')$, respectively. A dashed red arrow indicates delay or omission of all messages from round three onwards.

- $p_j = \ell$. This is the more complicated case, for which we leverage simulation based indistinguishability arguments. We continue below.

4.1 The $p_j = \ell$ case.

So far, we have obtained a pair of round three configurations C_3, C_3' that are AS-FFD in which the pivotal party is the leader ℓ. Recall that we arrived at these configurations by omitting some round one messages of the client c, and some round two messages of another party $p_j \neq \ell$. Beyond that, all other round one and round two messages were delivered in a timely manner.

Let us now partition the parties, apart from ℓ, p_i, c, into five sets V, W, X, Y, Z s.t. $|V| = |Z| = f - 1$, $|W| = |X| = |Y| = f - 2$.

- Consider first the following execution G_1. From configuration C_3, corrupt the $f-1$ parties of Z. The parties in Z send no further messages apart from round three messages to V, W, Y, ℓ, p_i. In particular, the parties in X do not hear from the parties in Z from round 3 onward. All other parties experience no further faults, and all their messages are delivered in a timely manner.

Denote by $\mathsf{D}(G_1)$ the decision value of this execution, note that G_1 can be obtained via corrupting at most f validators (p_i and all the parties in Z, recall that c is a client) with GST$= 0$. Thus by the good-case latency $\mathsf{T_{gc}} \leq 4$ assumption, all correct validators (V, W, X, Y, ℓ) decide a value after at most 2 rounds after configuration C_3. Assume w.l.o.g. that $\mathsf{FFV}(C_3) = 1$ (and hence $\mathsf{FFV}(C_3') = 0$). There are now two cases depending on $\mathsf{D}(G_1)$.

4.1.1 $\mathsf{D}(G_1) = 0 \neq \mathsf{FFV}(C_3)$.

The intuition. Apriori, this case might seem precarious. After all, we only have guarantees on the decision values of *failure free* extensions, namely $\mathsf{FF}(C_3), \mathsf{FF}(C_3')$. But G_1 is far from failure free. On closer inspection, however, one might notice that G_1 differs from $\mathsf{FF}(C_3)$ by de-facto omission failing the parties in Z. If such a deviation causes the decision value differ in $\mathsf{FF}(C_3)$ and G_1, then intuitively, some pivotal change is happening as the parties of Z experience an omission failure. Formally identifying and capturing this pivotal change allows us to move the proof forward by exhibiting a pair of round 4 configurations which are AS-FFD *without corrupting the leader* ℓ.

From that point, as there is only a single round left, the proof can be concluded in a similar fashion to Theorem 3.1. With that, we proceed to the the formal treatment of the case $\mathsf{FFV}(C_3) \neq \mathsf{D}(G_1)$.

Consider the following family of executions $G_1^i, i \in [f]$.

- G_1^i is constructed as follows: From configuration C_3, consider the first i of the $f-1$ parties of Z, denote this set by Z_i. The parties in Z_i have their round three messages omitted apart from messages to V, W, Y, ℓ, c, p_i. All other parties in Z experience no faults. In other words, all parties in Z_i experience an omission failure towards the parties in X in round 3, and experience no other faults.

Observe that $G_1^0 = \mathsf{FF}(C_3)$ and $\mathsf{D}(G_1^0) = \mathsf{FFV}(C_3)$ as no party experiences any faults. The proof now again splits into two cases, depending on whether $\mathsf{D}(G_1^{f-1}) \neq \mathsf{D}(G_1)$, or $\mathsf{D}(G_1^{f-1}) \neq \mathsf{D}(G_1^0)$, note that one of the two must hold as $\mathsf{FFV}(C_3) \neq \mathsf{D}(G_1)$.

If $\mathsf{D}(G_1^{f-1}) \neq \mathsf{D}(G_1)$, then we note that the round 4 configuration of G_1^{f-1} is, by definition of G_1^{f-1}, identical to the round 4 configuration of G_1. We thus obtain two executions, G_1^{f-1}, G_1 with different decision values, and in both of which ℓ *remains correct throughout the execution*, thus all correct validators decide at the end of the fourth round.

To not introduce overbearing notation, let us rename G_1^{f-1}, G_1 to G, G', respectively.

We can now define the following series of executions $G^i, i \in \{0, \ldots, f-1\}$.

- G^i is defined as follows. From G, consider the first i parties of Z, denoted by Z_i, and omit all their outgoing messages in round 4. All other round 4 and onwards messages are delivered without fault.

Note that $G^0 = G$, and G^{f-1} up to the end of round 4, is identical to G'. Since in both of G', G^{f-1} the leader ℓ remains correct, we have that all parties decide at the end of round 4, and thus $\mathsf{D}(G^{f-1}) = \mathsf{D}(G')$. Combined, we have that $\mathsf{D}(G^0) = \mathsf{D}(G) \neq \mathsf{D}(G') = \mathsf{D}(G^{f-1})$, and we in particular get that $\mathsf{D}(G^0) \neq \mathsf{D}(G^{f-1})$. Which then implies that there exists $i \in \{0, \ldots, f-2\}$ s.t. $\mathsf{D}(G^i) \neq \mathsf{D}(G^{i+1})$. Note now that both G^i, G^{i+1} are executions in which at most f parties are corrupted (p_i and at most $f-1$ parties of Z, recall that c is a client), and in particular, the leader ℓ isn't corrupted, and thus in both configurations all correct parties decide at the end of round 4, i.e. with no additional communication. Note that the difference between G^i, G^{i+1} is whether the $i+1$-th party in Z, denoted by z_{i+1} has its round 4 messages omitted. As such, by omitting all of z_{i+1}'s round 4 messages except for the message sent to an arbitrarily chosen correct victim validator p^* we obtain an agreement violation, as p^* decides according to G^i, while the rest of the validators decide according to G^{i+1}. This concludes the case of $\mathsf{D}(G_1^{f-1}) \neq \mathsf{D}(G_1)$.

We are thus left with the case that $\mathsf{D}(G_1^{f-1}) \neq \mathsf{D}(G_1^0)$. Which allows us to deduce that there exists $j \in \{0, \ldots, f-2\}$ s.t. $\mathsf{D}(G_1^j) \neq \mathsf{D}(G_1^{j+1})$. The difference between G_1^{j+1} and G_1^j is whether the $j+1$-th party in Z, denoted by z_{j+1} delivered its round 3 messages to X or had none of these delivered to any party in X. In an identical fashion to Lemma 3.8, we can interpolate between G_1^{j+1} and G_1^j via $|X|$ executions in depending on how many parties in X hear the round 3 message of z_{j+1}. This allows us to obtain AS-FFD round 4 configurations H, H' in which the pivotal party is $q \in X$. From here, we can conclude the proof via Lemma 3.9 (and corrupting q).

4.1.2 $\mathsf{D}(G_1) = 1 = \mathsf{FFV}(C_3)$

The previous section (Section 4.1.1) handled the case where $\mathsf{FFV}(G_1) \neq \mathsf{D}(G_1)$. In this section we deal with the case that $\mathsf{FFV}(C_3) = \mathsf{D}(G_1)$. Assuming this equality, we have that $1 = \mathsf{D}(G_1)$.

This is the case in which the proof proceeds in the spirit of the argument in [2]. We now define 4 additional executions G_2, G_3, G_4, G_5, each extending either C_3 or C_3'. Let us begin with G_5.

- G_5 is defined as follows. From configuration C_3', corrupt the $f-1$ parties of V. The parties in V send no further messages apart from round three messages to W, Y, Z, ℓ, p_i. In particular, the parties in X do not hear from the parties in V from round 3 onward. All other parties experience no further faults, and all their messages are delivered in a timely manner.

By a symmetric argument to the above case of G_1, we can conclude the proof if $\mathsf{D}(G_5) \neq \mathsf{FFV}(C_3')$. This leaves us with the case that $\mathsf{D}(G_5) = \mathsf{FFV}(C_3') = 0$. Next, we describe G_3.

- We corrupt the leader ℓ. It behaves in round three and four to the parties in V, W as in the failure free extension of C_3, and to the parties in Y, Z as in the failure free extension of C_3'. Note that this is well defined behavior, as the honest behavior of the leader in round four is a function of its state and the received messages in round 3, and the latter is identical in both C_3, C_3' as these configurations are AS-FFD with ℓ being pivotal. The $f-2$ parties in X are also corrupt, and crash after delivering all of their round three messages to all other correct parties (V, W, Y, Z).

By Agreement and termination, eventually all correct validators (V, W, Y, Z) decide a value. Next, we describe G_2.

- GST is sufficiently large. We corrupt the leader ℓ. The leader behaves as in the failure free extension of C_3 to V, W, X, and as in the failure free extension of C_3' to Z. The $f-2$ parties in Y are corrupted, they behave to X as in the failure free extension of C_3, and to all other parties as in the failure free extension of C_3'. X deliver their round three messages without fault, but the rest of their messages are delayed until after GST. All round three and on-wards messages from Z to X are delayed until after GST.

Note that in G_2, the parties in X, by the end of the fourth round, can not distinguish between G_1, G_2, and thus by the end of round 4 all validators in X decide as in $\mathsf{FFV}(C_3)$, i.e. 1. Finally, we describe G_4.

- GST is sufficiently large. We corrupt the leader ℓ, the leader behaves as in the failure free extension of C_3' to X, Y, Z, and as in the failure free extension of C_3 to V. The $f-2$ parties in W are corrupted, they behave to X as in the failure free extension of C_3', and to all other parties as in the failure free extension of C_3. X deliver their round three messages without fault, but the rest of their messages are delayed until after GST. All round three messages and on-wards from V to X are delayed until after GST.

Note that in G_4, the validators in X, by the end of the fourth round, can not distinguish between G_4, G_5, and thus by the end of round 4 all validators in X decide $\mathsf{FFV}(C_3')$, i.e. 0. Now notice that before GST, the validators in V, Z can not distinguish between executions G_2, G_3. Thus

by choosing GST to be sufficiently large, we get that the validators in V, Z decide the same value in both G_2, G_3. Since X output 1 in G_2, agreement implies that V, Z decide 1 in both G_2, G_3. A symmetric argument shows that V, Z decide the same value in both G_3, G_4. This leads to a contradiction since we established that X decides 0 in G_4, and thus V, Z decide 0 in G_4 and thus also in G_3, leading to a contradiction.

Note finally that in all the considered executions G_1, G_2, G_3, G_4, G_5 and the intermediate executions between the pair $\mathsf{FF}(C_3)$, G_1 and the intermediate executions between the pair $G_5, \mathsf{FF}(C_3')$, we corrupt at most f validators. The total number of validators in our construction is ℓ, p_i and the sizes of V, W, X, Y, Z, thus $n = 2 + 2(f-1) + 3(f-2) = 5f - 6$, as required. This concludes the proof. □

5 Protocols for synchrony and partial synchrony

In this section, we provide complementary upper bounds to our lower bound results in Theorem 3.1, and Theorem 4.1. Our protocols for CRBB are essentially a simplified version of the Multiple Concurrent Proposer [15] framework, adapted to the CRBB problem. More formally, we prove the following theorems.

Theorem 5.1. *Let f, n such that $3f < n$. Then, there exists an f-secure protocol Π for CRBB in partial synchrony with good-case latency $\mathsf{T}_{\mathsf{gc}} = 5$.*

Theorem 5.2. *Let f, n such that $5f < n$. Then, there exists an f-secure protocol Π for CRBB in partial synchrony with good-case latency $\mathsf{T}_{\mathsf{gc}} = 4$.*

Theorem 5.3. *Let f, n such that $2f < n$. Then, there exists an f-secure protocol Π for CRBB in synchrony with good-case latency $\mathsf{T}_{\mathsf{gc}} = 4$.*

In all protocols we present in this section, we make use of a blackbox BA protocol which we denote by Π_{BA}. Π_{BA} satisfies the standard agreement, termination, and validity properties, but need not have any round complexity guarantees. In our partial synchrony results, it can be instantiated by the protocol of [4], which is f-secure for any $f < \frac{n}{3}$. In our synchrony result, it can be instantiated by the protocol of [18], which is f-secure for any $f < \frac{n}{2}$.

Protocol 1: $\Pi_{\mathsf{CRBB},5}$

Instructions below are for validator p, unless explicitly stated otherwise (i.e. for clients). Denote by C the set of participating clients. Each correct client c has an input v_c. Each validator p initializes a vector o_p of length $m = |C|$, initialized to $\perp^m$.

1. **Upon** init: Each client c multicasts $\langle \mathsf{echo}, v_c \rangle_c$. Each validator p starts a local timer T_p.
2. **Upon** $T_p = \Delta$: Let E_p be the set of all echo messages received from clients. Multicast $\langle \mathsf{report}, E_p \rangle_p$.
3. **Upon** receiving $n - f$ report messages: Let C_p be the set of echo messages, received either directly or via a report messages, of all clients c for which the following holds.
   - p detected no equivocation from c, i.e. p did not observe two echo messages from c for 2 different values.
   - An echo message from c appears in $n - f$ of the report messages received by p.

Denote by E'_p be the set of all echo messages received from clients which appeared in at least $f+1$ report messages, including at most one echo message from each client, and breaking ties arbitrarily. If $p = \ell$: Multicast $\langle \mathsf{propose}, E'_\ell \rangle_\ell$.

4. **Upon** receiving propose message from ℓ or $T_p = 3\Delta$: If received propose message from ℓ, and $C_p \subseteq E'_\ell$, multicast $\langle \mathsf{vote}, E'_\ell \rangle_p$. Else, multicast $\langle \mathsf{vote}, \perp \rangle_p$.
5. **Wait until at least** $T = 4\Delta$**, and Upon** receiving $n-f$ vote messages: If received $n-f$ vote messages for some set E'_ℓ, denote them by $\mathcal{C}(\mathsf{vote}, E'_\ell)$, and multicast $\langle \mathsf{vote2}, E'_\ell, \mathcal{C}(\mathsf{vote}, E'_\ell) \rangle_p$. Else, multicast $\langle \mathsf{vote2}, \perp \rangle_p$.
6. **Upon** receiving $n-f$ vote2 messages:
   (a) **Early deciding condition:** If received $n-f$ valid vote2 messages for some set E'_ℓ, then decide E'_ℓ. Set $o_p \leftarrow E'_\ell$.
   (b) Else, if received at least one valid vote2 message for E'_ℓ, set $o_p \leftarrow E'_\ell$.
   (c) Else, set $o_p \leftarrow C_p$.

   Run m parallel instances of BA using Π_{BA}, each with a unique instance id, with input to instance i being $o_p[i]$. Decide the vector of outputs of the BA instances, if didn't decide before, and halt.

Proof of Theorem 5.1. Let f, n such that $3f < n$. Our protocol is given in $\Pi_{\mathsf{CRBB},5}$. Termination is clear by the termination property of Π_{BA}, as all validators halt once they terminate all m Π_{BA} instances, and due to the timeout in step 4 of $\Pi_{\mathsf{CRBB},5}$. We next tend to agreement.

Claim 5.4. $\Pi_{\mathsf{CRBB},5}$ *satisfies agreement against any* f*-bounded adversary.*

Proof. There are two cases. Either there exists a correct validator that decides by the early deciding condition, or not. The latter case is easy to handle, as then all correct validators decide by the output vector of the m instances of BA, for which agreement is guaranteed due to the agreement property of Π_{BA}. Otherwise, suppose some correct validator p decides a set S via the early deciding condition. A quorum intersection argument ($n - 2f > f$) implies that no correct validator can decide any other set via the early deciding condition. By protocol behavior, we have that if p decided a set S, then it received $n-f$ valid vote2 messages for S. This implies that all other correct validators received at least one valid vote2 message for S upon triggering step 6 of $\Pi_{\mathsf{CRBB},5}$. Note further, that a quorum intersection on the vote messages allows us to deduce that no valid vote2 message can exist for any other set S', as such a message must include a certificate of $n-f$ vote messages for S'. As such, we have that all correct validators q set their local vector variable o_q to be the indicator vector of S. This triggers the validity property of each of the m instances of Π_{BA}, which implies that all correct validators decide S, as required. □

Finally, we tend to censorship resistance and good-case latency.

Claim 5.5. $\Pi_{\mathsf{CRBB},5}$ *satisfies censorship-resistance against any* f*-bounded adversary. Furthermore,* $\Pi_{\mathsf{CRBB},5}$ *has good-case latency of* $\mathsf{T}_{\mathsf{gc}} = 5$.

Proof. Let c be a correct client with input value u, and assume $GST = 0$, then by the behavior of the protocol, all correct validators p have $\langle \mathsf{echo}, u \rangle_c \in C_p$. This in particular implies that a set E'_ℓ proposed by the leader can obtain a certificate of $n-f$ vote messages only if it contains $\langle \mathsf{echo}, u \rangle_c$.

Furthermore, this implies that all correct validators set $o_p[c] = u$ in step 6 of $\Pi_{\mathsf{CRBB},5}$, and so, by the validity property of Π_{BA}, $u \in S$ where S is the decided set of correct validators. For good-case latency, assume ℓ is correct, and $GST = 0$. The crucial observation is that for any correct validator p, if $c \in C_p$, then ℓ has seen $f+1$ report messages that contain an echo from c for *exactly* one value, as otherwise, $GST = 0$ would imply that all correct validators have seen an equivocation from c, and so $c \notin C_p$ for any correct validator p. As such, the leader's E'_ℓ passes the $C_p \subseteq E'_\ell$ check of all correct validators p, and so they all cast both vote and vote2 messages for E'_ℓ, as $GST = 0$, and so all correct validators decide E'_ℓ after 5 rounds. □

This concludes the proof of Theorem 5.1

□

Protocol 2: $\Pi_{\mathsf{CRBB},4}$

Setup. Each correct client c has an input v_c. Instructions for validator p, unless explicitly stated otherwise (i.e. instruction for client). Denote by C the number of participating clients. Each part initializes a vector o_p of length $m = |C|$, initialized to $\perp^m$.

1. **Upon** init: Each client c multicasts $\langle \mathsf{echo}, v_c \rangle_c$. Each validator p stars a local timer T_p.
2. **Upon** $T_p = \Delta$**:** Let E_p be the set of all echo messages received from clients. Multicast $\langle \mathsf{report}, E_p \rangle_p$.
3. **Upon** receiving $n - f$ report messages: Let C_p be the set of echo messages of all clients c for which the following holds.
   - p detected no equivocation from c, p did not observe two echo messages from c for 2 different values.
   - An echo message from c appears in $n - f$ of the report messages received by p.

   Denote by E'_p be the set of all echo messages received from clients which appeared in at least $f + 1$ report messages, including at most one echo message from each client, and breaking ties arbitrarily. If $p = \ell$: Multicast $\langle \mathsf{propose}, E'_\ell \rangle_\ell$.
4. **Upon** receiving propose message from ℓ or $T_p = 3\Delta$: If received propose message from ℓ, and $C_p \subseteq E'_\ell$, multicast $\langle \mathsf{vote}, E'_\ell \rangle_p$. Else, multicast $\langle \mathsf{vote}, \perp \rangle_p$.
5. **Upon** receiving $n - f$ vote messages:
   (a) **Early deciding condition:** If received $n - f$ vote messages for some set E'_ℓ, then decide E'_ℓ and set $o_p \leftarrow E'_\ell$.
   (b) Else, if received $n - 3f$ vote messages for some E'_ℓ, then set $o_p \leftarrow E'_\ell$, breaking ties arbitrarily.
   (c) Else, set $o_p \leftarrow C_p$.

   Run m parallel instances of BA using Π_{BA}, each with a unique session id, with input to instance i being $o_p[i]$. Decide the vector of outputs of the BA instances, if didn't decide before, and halt.

Proof of Theorem 5.2. Let f, n such that $5f < n$. Our protocol is given in $\Pi_{\mathsf{CRBB},4}$. Termination is clear by the termination property of Π_{BA}, as all validators halt once they terminate all m Π_{BA} instances, and due to the timeout in step 4 of $\Pi_{\mathsf{CRBB},4}$. We next tend to agreement.

Claim 5.6. $\Pi_{\mathsf{CRBB},4}$ *satisfies agreement against any f-bounded adversary.*

Proof. There are two cases. Either there exists a correct validator that decides by the early deciding condition, or not. The latter case is easy to handle, as then all correct validators decide by the output vector of the m instances of BA, for which agreement is guaranteed due to the agreement property of Π_{BA}. Otherwise, suppose some correct validator p decides a set S via the early deciding condition. A quorum intersection argument ($n - 2f > 3f > f$) implies that no correct validator can decide any other set via the early deciding condition. By protocol behavior, we have that if p decided a set S, then it received $n - f$ vote messages for S, this implies that all other correct validators received at least $n - 3f$ vote message for S. Note further that a quorum intersection argument ($n - 4f > f$) implies that no correct validator received $n - 3f$ vote messages for any other set. This implies that upon triggering step 5 of $\Pi_{\mathsf{CRBB},4}$, all correct validators q set their local variable q to be the indicator vector of S. This triggers the validity property of each of the m instances of Π_{BA}, which implies that all correct validators decide S, as required. □

Finally, we tend to censorship resistance and good-case latency.

Claim 5.7. *$\Pi_{\mathsf{CRBB},4}$ satisfies censorship-resistance against any f-bounded adversary. Furthermore, $\Pi_{\mathsf{CRBB},4}$ has good-case latency of $\mathsf{T}_{\mathsf{gc}} = 4$.*

Proof. Let c be a correct client with input value u, and assume $GST = 0$, then by the behavior of the protocol, all correct validators p have $\langle \mathsf{echo}, u \rangle_c \in C_p$. This in particular implies that a set E'_ℓ proposed by the leader can obtain $n - 3f$ vote messages only if it contains $\langle \mathsf{echo}, u \rangle_c$. Thus, in step 5 of $\Pi_{\mathsf{CRBB},4}$, all correct validators set $o_p[c] = u$, regardless of the triggered condition. By the validity property of Π_{BA}, we thus have that $u \in S$, where S is the decided set of correct validators. For good-case latency, assume ℓ is correct, and $GST = 0$. Observe that for any correct validator p, if a client $c \in C_p$, then ℓ has seen at least $f + 1$ report messages that contain an echo from c for *exactly* one value, as otherwise, $GST = 0$ implies that all correct validators have seen an equivocation from c, and so $c \notin C_p$ for any correct validator p. As such, the leader's proposal E'_ℓ passed the $C_p \subseteq E'_\ell$ check of all correct validators p, and so they all cast vote messages for E'_ℓ. Since $GST = 0$, this implies that all correct validators see $n - f$ vote messages for E'_ℓ, and thus decide after 4 rounds. □

This concludes the proof of Theorem 5.2. □

Protocol 3: $\Pi_{\mathsf{CRBB},4,\mathsf{Sync}}$

Setup. Each correct client c has an input v_c. Instructions for validator p, unless explicitly stated otherwise (i.e. instruction for client). Denote by C the number of participating clients. Each part initializes a vector o_p of length $m = |C|$, initialized to $\perp^m$.

1. **Round** $t = 0$: Each client c multicasts $\langle \mathsf{echo}, v_c \rangle_c$.
2. **Round** $t = \Delta$: Let E_p be a set of all echo messages received from clients. Multicast $\langle \mathsf{report}, E_p \rangle_p$.
3. **Round** $t = 2\Delta$: Let C_p be the set of echo messages of all clients c for which the following holds.
   - p detected no equivocation from c, p did not observe two echo messages from c for 2 different values.
   - An echo message from c appears in $n - f$ of the report messages received by p.

Denote by E'_p be the set of all echo messages received from clients which appeared in at least one report messages, including at most one echo message from each client, and breaking ties arbitrarily. If $p = \ell$: Multicast $\langle \mathsf{propose}, E'_\ell \rangle_\ell$.

4. **Round** $t = 3\Delta$: If received propose message from ℓ, and for every client $c \in C_p$, E'_ℓ contains an echo message from c, multicast $\langle \mathsf{vote}, E'_\ell \rangle_p$. Else, multicast $\langle \mathsf{vote}, \perp \rangle_p$.
5. **Round** $t = 4\Delta$: **Early deciding condition:** If received $n - f$ vote messages for some set E'_ℓ, and detected no equivocation from ℓ: Decide E'_ℓ and set $o_p \leftarrow E'_\ell$. Denote by V_p a certificate of the $n - f$ vote messages for E'_ℓ, multicast $\langle \mathsf{decide}, E'_\ell, V_p \rangle_p$.
6. **Round** $t \geq 5\Delta$: If received a valid decide message for some E'_ℓ, then set $o_p \leftarrow E'_\ell$, breaking ties arbitrarily. Else, set $o_p \leftarrow C_p$. Run m parallel instances of BA using Π_{BA}, each with a unique session id, commencing at round $t = 5\Delta$, with input to instance i being $o_p[i]$. Decide the vector of outputs of the BA instances, if didn't decide before, and halt.

Proof of Theorem 5.3. Let f, n such that $2f < n$. Our protocol is given in $\Pi_{\mathsf{CRBB,4,Sync}}$. Termination is clear by the termination property of Π_{BA}, as all validators halt once they terminate all m Π_{BA} instances. We next tend to agreement.

Claim 5.8. $\Pi_{\mathsf{CRBB,4,Sync}}$ *satisfies agreement against any f-bounded adversary.*

Proof. There are two cases. Either there exists a correct validator that decides by the early deciding condition, or not. The latter case is easy to handle, as then all correct validators decide by the output vector of the m instances of BA, for which agreement is guaranteed due to the agreement property of Π_{BA}. Otherwise, suppose some correct validator p decides a set S via the early deciding condition. By the behavior of the protocol, we have that p received $n - f$ valid vote messages for S, and detected no equivocation from ℓ. In particular, this implies that no correct validator send a vote message for any other set $S' \neq S$, as that would reveal to p an equivocation from ℓ in its propose message. This implies that for any set $S' \neq S$. Any other correct validator q received at most $f < n - f$ valid vote messages for S', thus the only set a correct validator can decide via the early deciding condition is S. Furthermore, as there are at most $f < n - f$ valid vote messages for any set $S' \neq S$, this implies that in round $t = 5\Delta$, all correct validators q set $o_q \leftarrow S$, as they receive a valid decide message from p for S, and cant receive a valid (i.e. a decide message containing a vote certificate) for any other set $S' \neq S$. This triggers the validity property of each of the m instances of Π_{BA}, which implies that all correct validators decide S, as required. □

Finally, we tend to censorship resistance and good-case latency.

Claim 5.9. $\Pi_{\mathsf{CRBB,4,Sync}}$ *satisfies censorship resistance against any f-bounded adversary, Furthermore, $\Pi_{\mathsf{CRBB,4,Sync}}$ has good-case latency of $\mathsf{T}_{\mathsf{gc}} = 4$.*

Proof. Let c be a correct client with input value u. Then by the behavior of the protocol, all correct validators p have $\langle \mathsf{echo}, u \rangle_c \in C_p$. This in particular implies that a set E'_ℓ proposed by the leader can obtain $n - f$ vote messages only if it contains $\langle \mathsf{echo}, u \rangle_c$. Thus, any valid decide message for a set S' received by a correct validator at round $t = 5\Delta$ must contain $\langle \mathsf{echo}, u \rangle_c$, and so all correct validators set $o_p[c] = u$. By the validity property of Π_{BA}, we thus have that $u \in S$, where S is the set decided by correct validators. For good-case latency, assume ℓ is correct. Observe that for any correct validator p, if a client $c \in C_p$, then ℓ has seen an echo message from c as it was included in at

least one report message of a correct validator. This implies, by the instructions of $\Pi_{\mathsf{CRBB},4,\mathsf{Sync}}$ in round $t = 2\Delta$, that ℓ includes an echo message from c in its proposed set E'_ℓ. As this holds for any client $c \in C_p$ for all correct validators p, we have that all correct validators at round $t = 3\Delta$, cast a vote message for E'_ℓ, and so all correct validators at round $t = 4\Delta$ see $n - f$ valid vote messages for E'_ℓ, and detect no equivocation from ℓ. Thus all correct validators decide E'_ℓ after 4 rounds, as required. □

This concludes the proof of Theorem 5.3. □

Acknowledgments. We thank Pranav Garimidi, Kartik Nayak, Joachim Neu, Max Resnick, Tim Roughgarden, and Giulia Scaffino for fruitful discussions. This work was carried out while Yuval was affiliated with a16z Crypto Research and Ittai Abraham and Ling Ren were visiting a16z Crypto Research.

References

[1] Ittai Abraham, Dahlia Malkhi, Kartik Nayak, Ling Ren, and Maofan Yin. Sync HotStuff: Simple and practical synchronous state machine replication. In *SP*, pages 106–118. IEEE, 2020.

[2] Ittai Abraham, Kartik Nayak, Ling Ren, and Zhuolun Xiang. Good-case latency of byzantine broadcast: a complete categorization. In *PODC*, pages 331–341. ACM, 2021.

[3] Maryam Bahrani, Pranav Garimidi, and Tim Roughgarden. Transaction fee mechanism design in a post-mev world. In *AFT*, volume 316 of *LIPIcs*, pages 29:1–29:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.

[4] Ethan Buchman, Jae Kwon, and Zarko Milosevic. The latest gossip on BFT consensus. arXiv:1807.04938v3 [cs.DC], 2018. URL: http://arxiv.org/abs/1807.04938v3, arXiv:1807.04938v3.

[5] Vitalik Buterin. The problem of censorship, 2015. URL: https://blog.ethereum.org/2015/06/06/the-problem-of-censorship.

[6] Vitalik Buterin. State of research: increasing censorship resistance of transactions under proposer/builder separation (pbs), 2021. URL: https://notes.ethereum.org/@vbuterin/pbs_censorship_resistance.

[7] Miguel Castro and Barbara Liskov. Practical Byzantine fault tolerance. In *OSDI*, pages 173–186. USENIX Association, 1999.

[8] Miguel Castro and Barbara Liskov. Practical Byzantine Fault Tolerance and Proactive Recovery. *ACM Transactions on Computer Systems*, 20(4), 2002.

[9] Benjamin Y. Chan and Rafael Pass. Simplex consensus: A simple and fast consensus protocol. In *TCC (4)*, volume 14372 of *Lecture Notes in Computer Science*, pages 452–479. Springer, 2023.

[10] Brendan Kobayashi Chou, Andrew Lewis-Pye, and Patrick O'Grady. Minimmit: Fast finality with even faster blocks. *CoRR*, abs/2508.10862, 2025.

[11] Yuval Efron, Joachim Neu, Ling Ren, and Ertem Nusret Tas. Optimal good-case latency for sleepy consensus. *IACR Cryptol. ePrint Arch.*, page 1856, 2025.

[12] Elijah Fox, Mallesh M. Pai, and Max Resnick. Censorship resistance in on-chain auctions. In *AFT*, volume 282 of *LIPIcs*, pages 19:1–19:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023.

[13] Francesco D'Amato. Forward inclusion list, 2022. URL: https://notes.ethereum.org/@fradamt/H1ZqdtrBF.

[14] Francesco D'Amato. Pbs censorship-resistance alternatives, 2022. URL: https://notes.ethereum.org/@fradamt/H1TsYRfJc.

[15] Pranav Garimidi, Joachim Neu, and Max Resnick. Multiple concurrent proposers: Why and how. *CoRR*, abs/2509.23984, 2025.

[16] Quentin Kniep, Kobi Sliwinski, and Roger Wattenhofer. Solana alpenglow consensus, increased bandwidth, reduced latency. 2025. URL: https://www.anza.xyz/blog/alpenglow-a-new-consensus-for-solana.

[17] Jean-Philippe Martin and Lorenzo Alvisi. Fast byzantine consensus. *IEEE Trans. Dependable Secur. Comput.*, 3(3):202–215, July 2006. doi:10.1109/TDSC.2006.35.

[18] Atsuki Momose and Ling Ren. Optimal communication complexity of authenticated byzantine agreement. In *DISC*, volume 209 of *LIPIcs*, pages 32:1–32:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021.

[19] Mike Neuder. No free lunch – a new inclusion list design, 2023. URL: https://ethresear.ch/t/no-free-lunch-a-new-inclusion-list-design/16389.

[20] Victor Shoup, Jakub Sliwinski, and Yann Vonlanthen. Kudzu: Fast and simple high-throughput BFT. In *DISC*, volume 356 of *LIPIcs*, pages 42:1–42:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025.

[21] Nibesh Shrestha, Aniket Kate, and Kartik Nayak. Hydrangea: Optimistic two-round partial synchrony with one-third fault resilience. *IACR Cryptol. ePrint Arch.*, page 1112, 2025.

[22] Thomas Thiery, Barnabe Monnot, Francesco D'Amato, and Julian Ma. Fork-choice enforced inclusion lists (focil): A simple committee-based inclusion list proposal, 2024.

[23] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan-Gueta, and Ittai Abraham. Hotstuff: BFT consensus with linearity and responsiveness. In *PODC*, pages 347–356. ACM, 2019.